MIPT & YSDA · AUTUMN 2026

Deep Generative Models

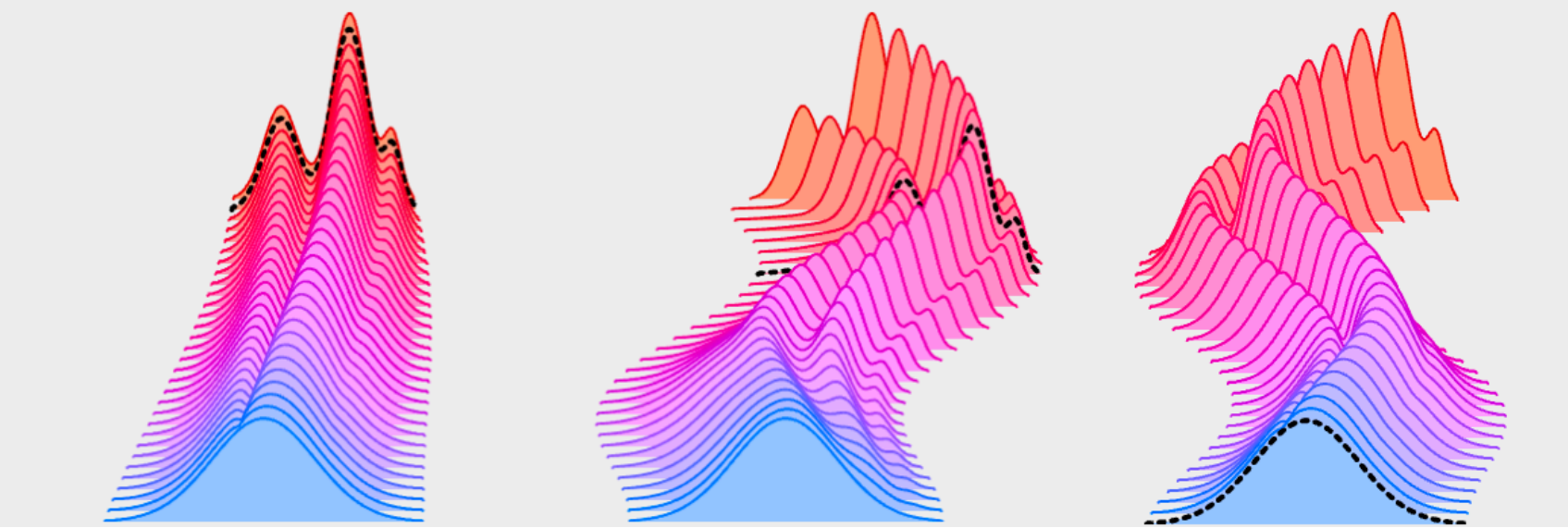
Lecture 12

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$



Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lecture

Problem: The true vector field $\mathbf{v}(\mathbf{x}, t)$ is **unknown**.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$

$$\frac{\partial p_t(\mathbf{x}|\mathbf{z})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, \mathbf{z}, t)p_t(\mathbf{x}|\mathbf{z})\right).$$

Recap of Previous Lecture

Conditional Flow Matching (CFM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

If $\operatorname{supp}(p_t(\mathbf{x}))=\mathbb{R}^m$, the FM and CFM objectives differ by a constant independent of $\boldsymbol{\theta}$, so they have the same minimizers.

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lecture

Training

1. Sample $t \sim U[0, 1]$, $\mathbf{z} \sim p(\mathbf{z})$, $\mathbf{x}_t \sim p_t(\mathbf{x}|\mathbf{z})$.
2. Compute loss $\mathcal{L} = \|\mathbf{v}(\mathbf{x}_t, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2$.

Sampling

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \boldsymbol{\psi}_1(\mathbf{x}_0) = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1).$$

Recap of Previous Lectures

Theorem

The following vector field generates the probability path $p_t(\mathbf{x})$:

$$\mathbf{v}(\mathbf{x}, t) = \mathbb{E}_{p_t(\mathbf{z}|\mathbf{x})}\mathbf{v}(\mathbf{x}, \mathbf{z}, t) = \int \mathbf{v}(\mathbf{x}, \mathbf{z}, t)\frac{p_t(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_t(\mathbf{x})}d\mathbf{z}$$

Flow Matching (FM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

Recap of Previous Lectures

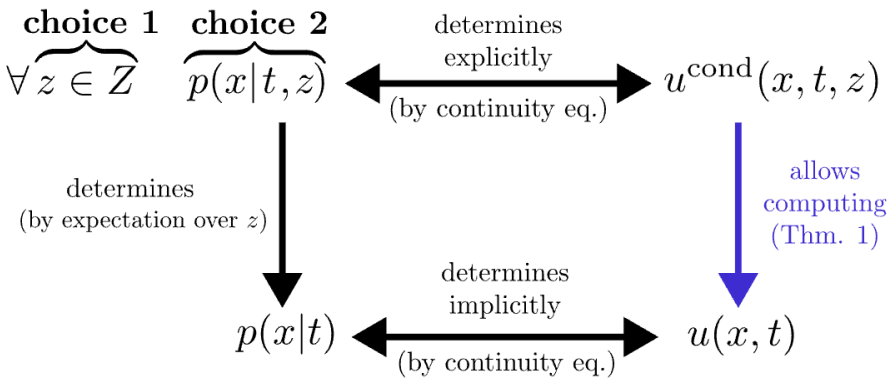
Conditional Flow Matching (CFM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Theorem

If $\operatorname{supp}(p_t(\mathbf{x}))=\mathbb{R}^m$, the FM and CFM objectives differ by a constant independent of $\boldsymbol{\theta}$, so they have the same minimizers.

Recap of Previous Lecture



Open Questions

- **Q1** How should we choose the convenient conditioning latent variable $\mathbf{z}$?
- **Q2** How can we parametrize $p_t(\mathbf{x})$ (or $p_t(\mathbf{x}|\mathbf{z})$) to enforce the following constraints?

$$\mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0, \mathbf{I}); \quad \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}).$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right);$$
$$\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right);$$
$$\mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

$$\begin{aligned}\mathbf{v}(\mathbf{x}_t, \mathbf{z}, t) &= \boldsymbol{\psi}_t'\left(\boldsymbol{\psi}_t(\mathbf{x}_t, \mathbf{z})^{-1}, \mathbf{z}\right) \\ &= \boldsymbol{\mu}_t'(\mathbf{z}) + \frac{\boldsymbol{\sigma}_t'(\mathbf{z})}{\boldsymbol{\sigma}_t(\mathbf{z})} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{z}))\end{aligned}$$

Recap of Previous Lecture

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z} = \mathbf{x}_1$. Then $p(\mathbf{z}) = p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{x}_1)p_1(\mathbf{x}_1)d\mathbf{x}_1$$

We need to ensure the boundary constraints:

$$\begin{cases} \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0, \mathbf{I}) \\ \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}) \end{cases} \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \\ p_1(\mathbf{x}|\mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

Recap of Previous Lecture

$$p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \quad p_1(\mathbf{x}|\mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_1)\right);$$
$$\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_1) = t\mathbf{x}_1 \\ \boldsymbol{\sigma}_t(\mathbf{x}_1) = 1 - t \end{cases} \Rightarrow \begin{cases} p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2 \cdot \mathbf{I}\right) \\ \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

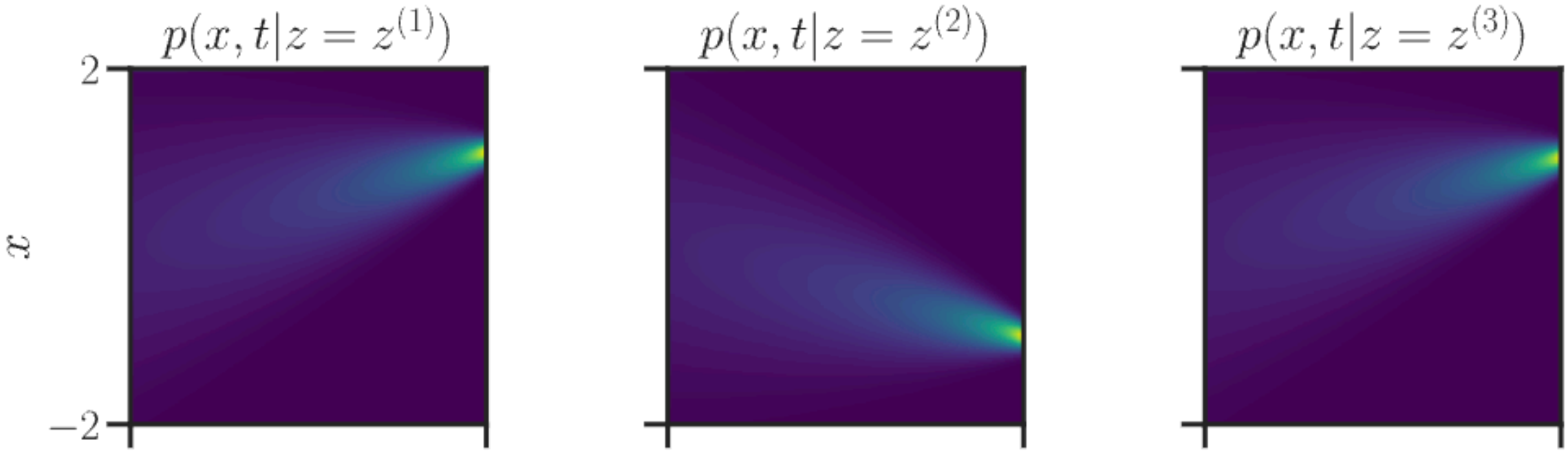


image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Rectified Flow Objective

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t} = \mathbf{x}_1 - \mathbf{x}_0$$

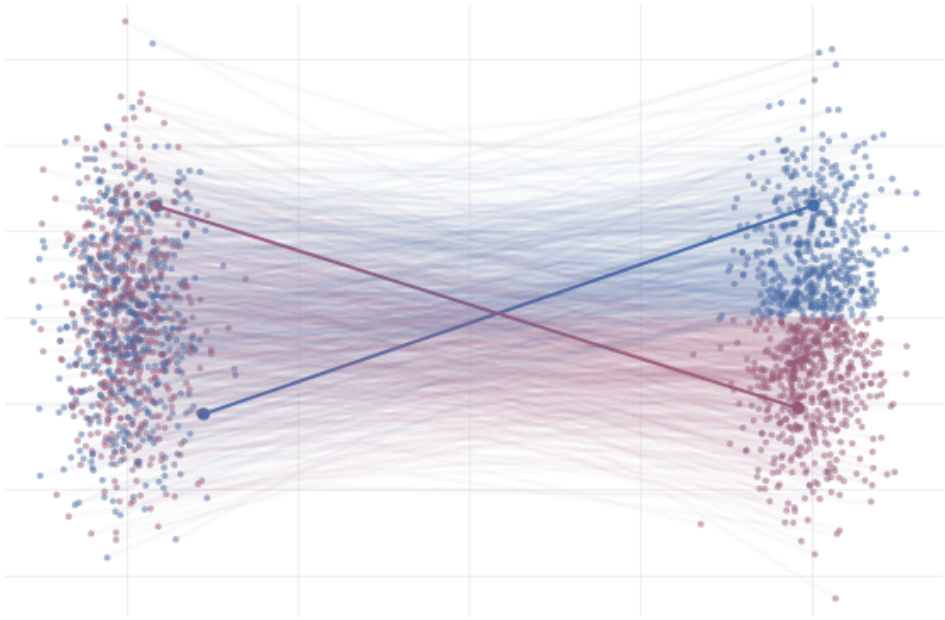
$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})} \left\|\mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2$$

$$= \mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})} \left\|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, t\right)\right\|^2$$

image credit: https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html
Liu X., Gong C., Liu Q. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, 2022

Recap of Previous Lecture

- $\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t)$ defines straight lines between $p_{\text{data}}(\mathbf{x})$ and $\mathcal{N}(0, \mathbf{I})$.
- **Straight training paths do not imply straight sampling trajectories.**



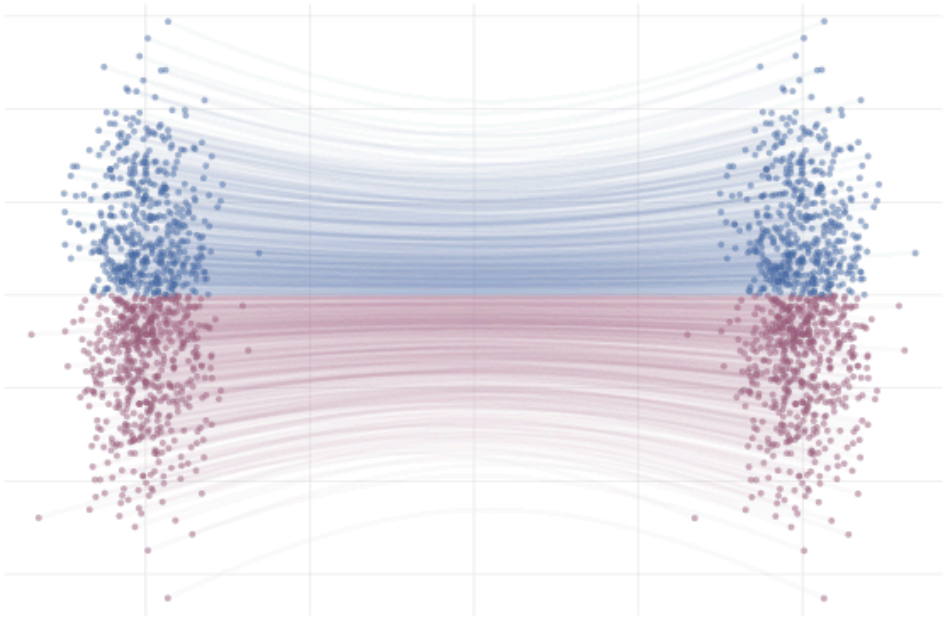


image credit: https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html
Liu X., Gong C., Liu Q. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, 2022

Recap of Previous Lecture

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})}\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Training

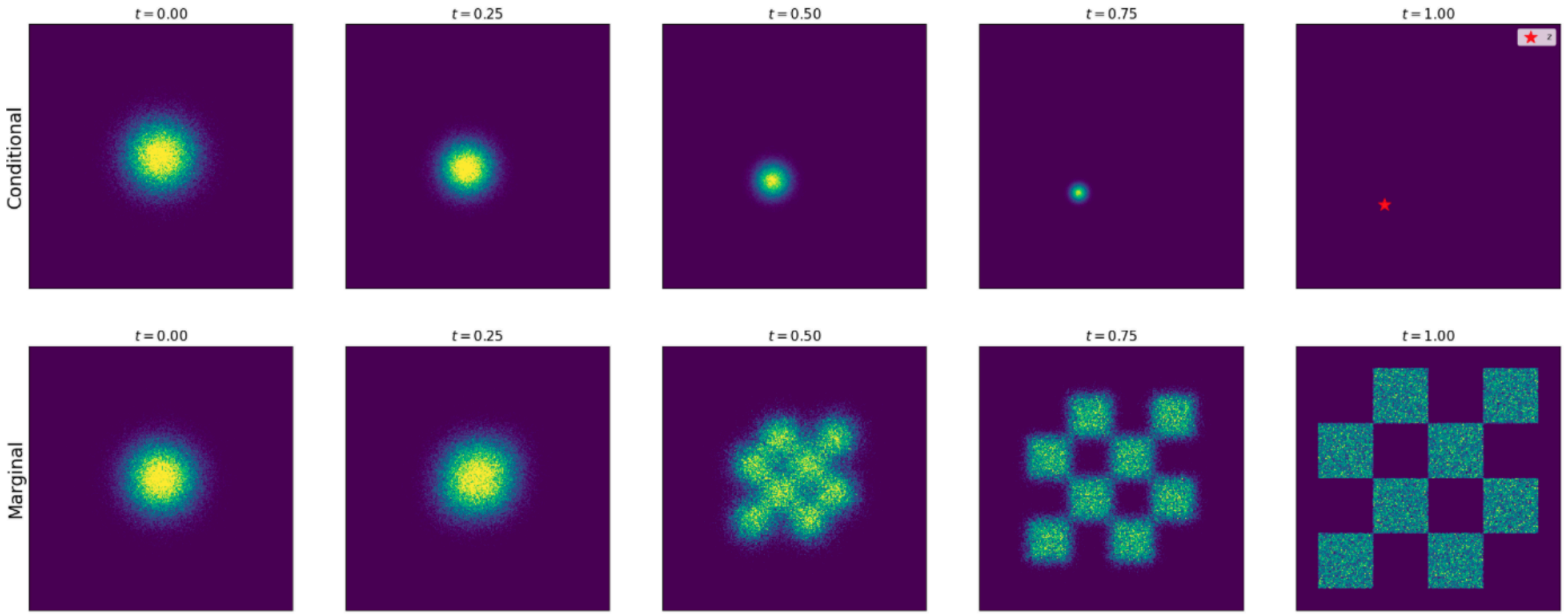
1. Sample $\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})$, $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$, $t\sim U[0,1]$.
2. Compute noisy image $\mathbf{x}_t=t\mathbf{x}_1+(1-t)\mathbf{x}_0$.
3. Compute loss $\mathcal{L}=\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2$.

Sampling

1. Sample $\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1=\boldsymbol{\psi}_1(\mathbf{x}_0)=\texttt{ODESolve}_v(\mathbf{x}_0,\boldsymbol{\theta},t_0=0,t_1=1).$$

Recap of Previous Lecture



- One-sided conditioning gives us the way to construct a generative model.
- Now we extend it to image-to-image formulation (mapping between two distinct data distributions $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$).

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Pair Conditioning

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2\to\min_{\boldsymbol{\theta}}$$

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z}=(\mathbf{x}_0,\mathbf{x}_1)$. Then $p(\mathbf{z})=p(\mathbf{x}_0,\mathbf{x}_1)=p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x})=\int p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)d\mathbf{x}_0d\mathbf{x}_1$$

Pair Conditioning

We must enforce boundary constraints:

$$\begin{cases} p_0(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}); \\ p_1(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) \end{cases}$$

Pair Conditioning

We must enforce boundary constraints:

$$\begin{cases} p_0(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}); \\ p_1(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) \end{cases} \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

Two-Sided Conditioning

$$p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)\right);$$

$$\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

Let’s consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

Let’s consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases} \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

Let’s consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases} \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

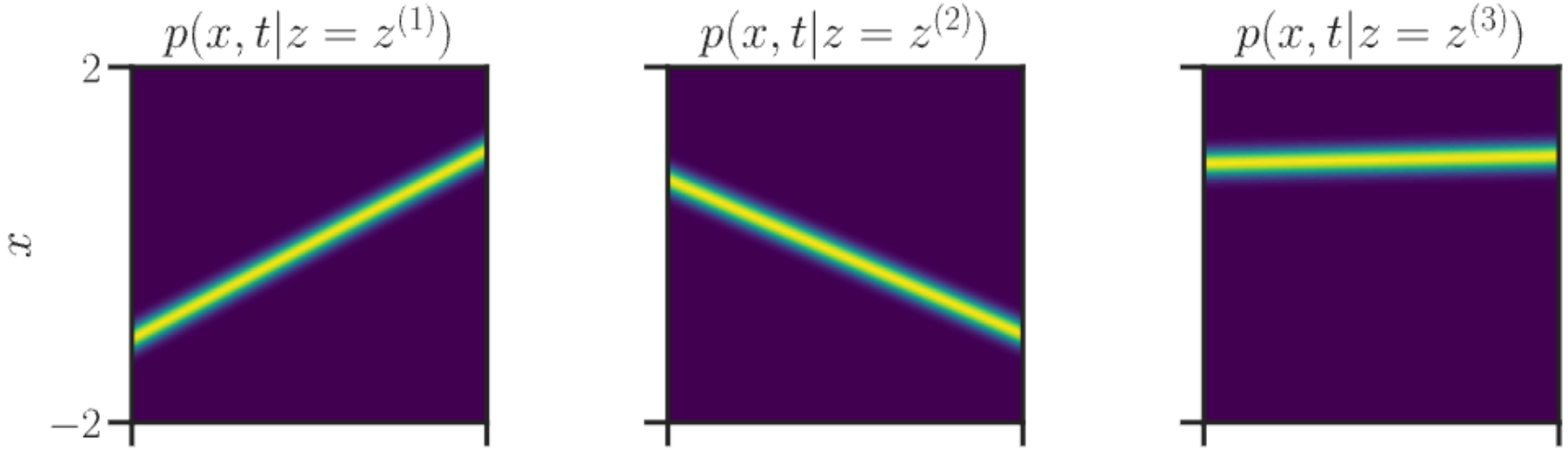


image credit: A Visual Dive into Conditional Flow Matching

Flow Matching: One-Sided vs. Two-Sided Conditioning

$$\mathbf{z} = \mathbf{x}_1$$
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

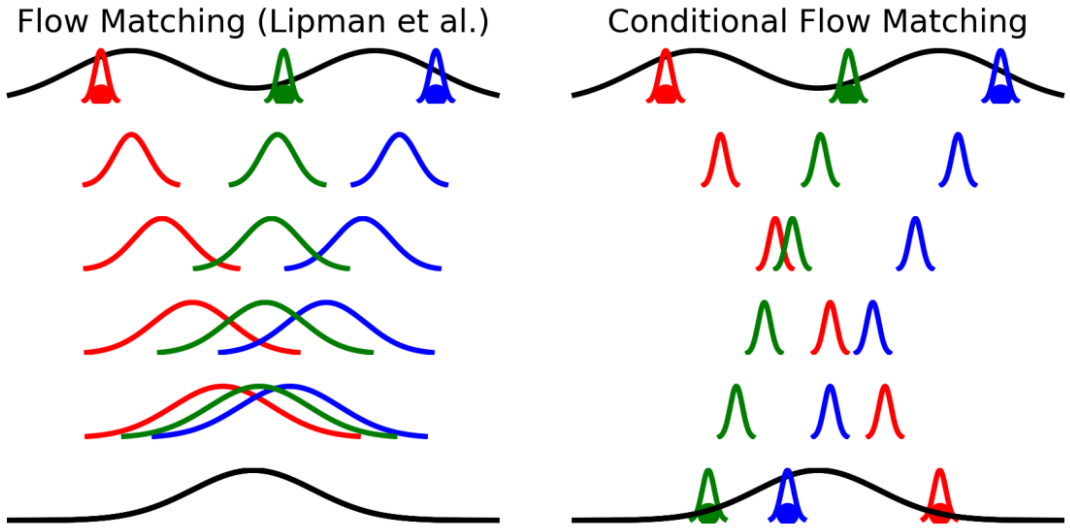
$$\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$$
$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Flow Matching: One-Sided vs. Two-Sided Conditioning

$$\mathbf{z} = \mathbf{x}_1$$
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$$
$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$



Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right);$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Conditional Vector Field

$$\begin{aligned}\frac{d\mathbf{x}_t}{dt} &= \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) \\ &= \boldsymbol{\mu}'_t(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))\end{aligned}$$

Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right);$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Conditional Vector Field

$$\begin{aligned}\frac{d\mathbf{x}_t}{dt} &= \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) \\ &= \boldsymbol{\mu}_t'(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}_t'(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))\end{aligned}$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \mathbf{x}_1 - \mathbf{x}_0$$

Two-Sided Conditioning: Vector Field and Objective

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2$$
$$=\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{(\mathbf{x}_0,\mathbf{x}_1)\sim p(\mathbf{x}_0,\mathbf{x}_1)}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)}\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2$$

Two-Sided Conditioning: Vector Field and Objective

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2$$
$$=\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{(\mathbf{x}_0,\mathbf{x}_1)\sim p(\mathbf{x}_0,\mathbf{x}_1)}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)}\left\|(\mathbf{x}_1-\mathbf{x}_0)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2$$

- This yields the same procedure as for one-sided conditioning!
- Now, we do not require that $p_0(\mathbf{x})$ is necessarily $\mathcal{N}(0,\mathbf{I})$.

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It’s possible to apply this approach to paired tasks, e.g., style transfer.

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It's possible to apply this approach to paired tasks, e.g., style transfer.

Training

1. Sample $(\mathbf{x}_0, \mathbf{x}_1) \sim p(\mathbf{x}_0, \mathbf{x}_1)$, $t \sim U[0, 1]$.
2. Compute noisy image $\mathbf{x}_t = t\mathbf{x}_1 + (1 - t)\mathbf{x}_0$.
3. Compute loss $\mathcal{L} = \|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2$.

CFM with Two-Sided Conditioning: Training and Sampling

Sampling

1. Sample $\mathbf{x}_0 \sim p_0(\mathbf{x})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \boldsymbol{\psi}_1(\mathbf{x}_0) = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1).$$

Stable Diffusion 3: Scalable Flow Matching



Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Score-Based Generative Models through SDEs

Training Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Score-Based Generative Models through SDEs

Training Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Variance Exploding SDE (NCSN)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0),\left[\sigma^2(t)-\sigma^2(0)\right]\cdot\mathbf{I}\right),\quad \sigma(0)=0$$

Variance Preserving SDE (DDPM)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0)\alpha(t),\left(1-\alpha(t)^2\right)\cdot\mathbf{I}\right);$$
$$\alpha(t)=e^{-\frac{1}{2}\int_0^t\beta(s)ds}$$

Score-Based Generative Models through SDEs

Flow matching uses reverse time direction:

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

Tweedie's Formula

Recall Lecture 6, now with a scaled signal: for $\alpha_t > 0$, $\sigma_t > 0$,

$$p_t(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\alpha_t \mathbf{x}_1, \sigma_t^2 \mathbf{I}) \quad \Rightarrow \quad \mathbb{E}[\mathbf{x}_1 \mid \mathbf{x}_t] = \frac{\mathbf{x}_t + \sigma_t^2 \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)}{\alpha_t}.$$

When the conditional velocity is affine in $\mathbf{x}_1$, averaging over $\mathbf{x}_1 \mid \mathbf{x}_t$ expresses the marginal velocity through the score.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022
Efron B. Tweedie's Formula and Selection Bias, 2011

Score-Based Generative Models through SDEs

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

$$\textbf{VE (NCSN): } p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma^2_{1-t} \cdot \mathbf{I}\right)$$

$$\textbf{VP (DDPM): } p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, \left(1 - \alpha^2_{1-t}\right) \cdot \mathbf{I}\right)$$

Score-Based Generative Models through SDEs

Flow Matching Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}_t'(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}_t'(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Score-Based Generative Models through SDEs

Flow Matching Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}_t'(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}_t'(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Let’s derive the conditional vector fields for VE (NCSN) and VP (DDPM).

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Variance Exploding SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \mathbf{I}\right) \quad \Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = -\frac{\sigma'_{1-t}}{\sigma_{1-t}}(\mathbf{x}_t - \mathbf{x}_1)$$

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Variance Exploding SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \mathbf{I}\right) \quad \Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = -\frac{\sigma'_{1-t}}{\sigma_{1-t}}(\mathbf{x}_t - \mathbf{x}_1)$$

Variance Preserving SDE Probability Path

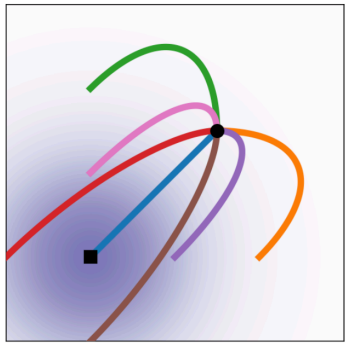
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, (1 - \alpha_{1-t}^2)\mathbf{I}\right)$$

$$\Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\alpha'_{1-t}}{1 - \alpha_{1-t}^2} \cdot (\alpha_{1-t}\mathbf{x}_t - \mathbf{x}_1)$$

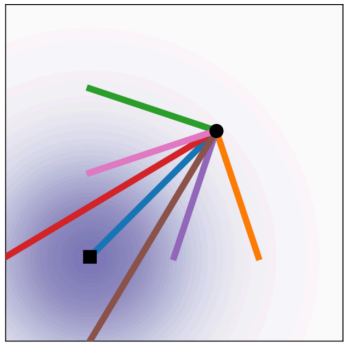
Thus, VE/VP SDE models correspond to particular choices of the Gaussian probability path within the flow matching framework.

Flow Matching vs Score-Based SDE Models

Trajectories



Diffusion

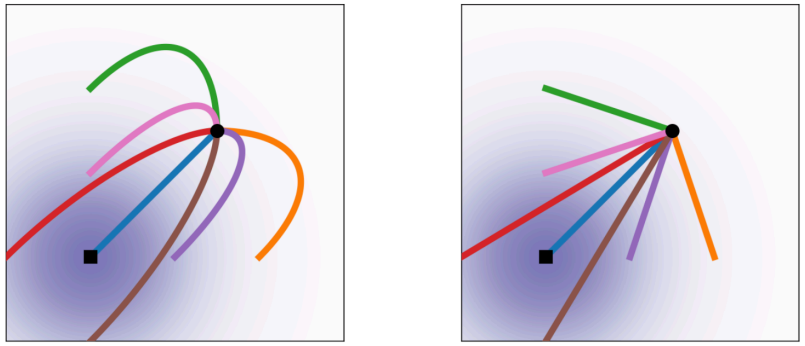


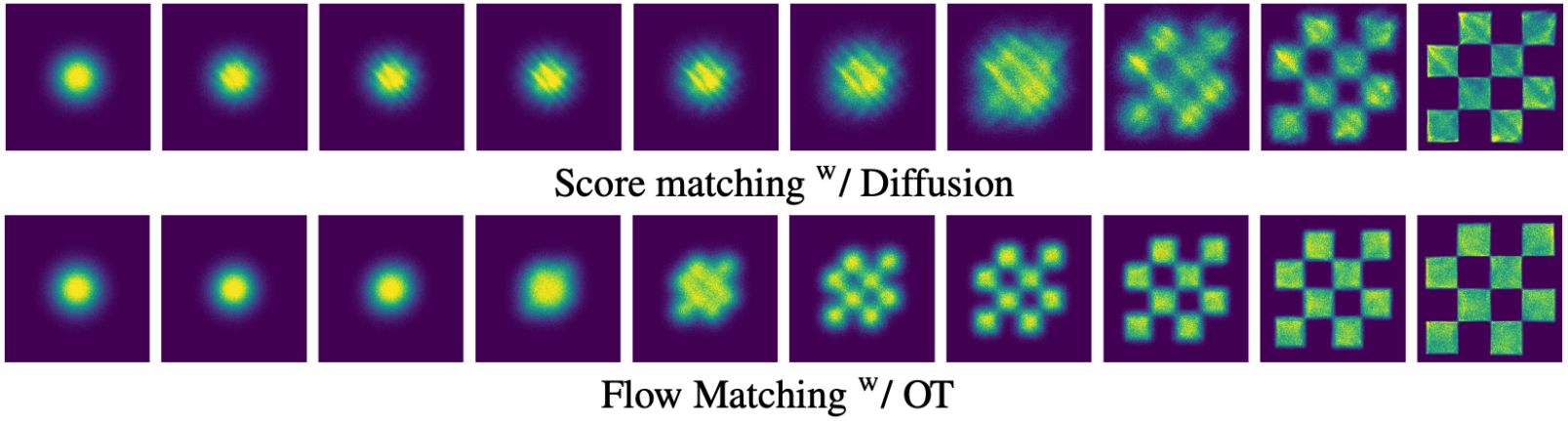
OT

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Flow Matching vs Score-Based SDE Models

Trajectories





Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

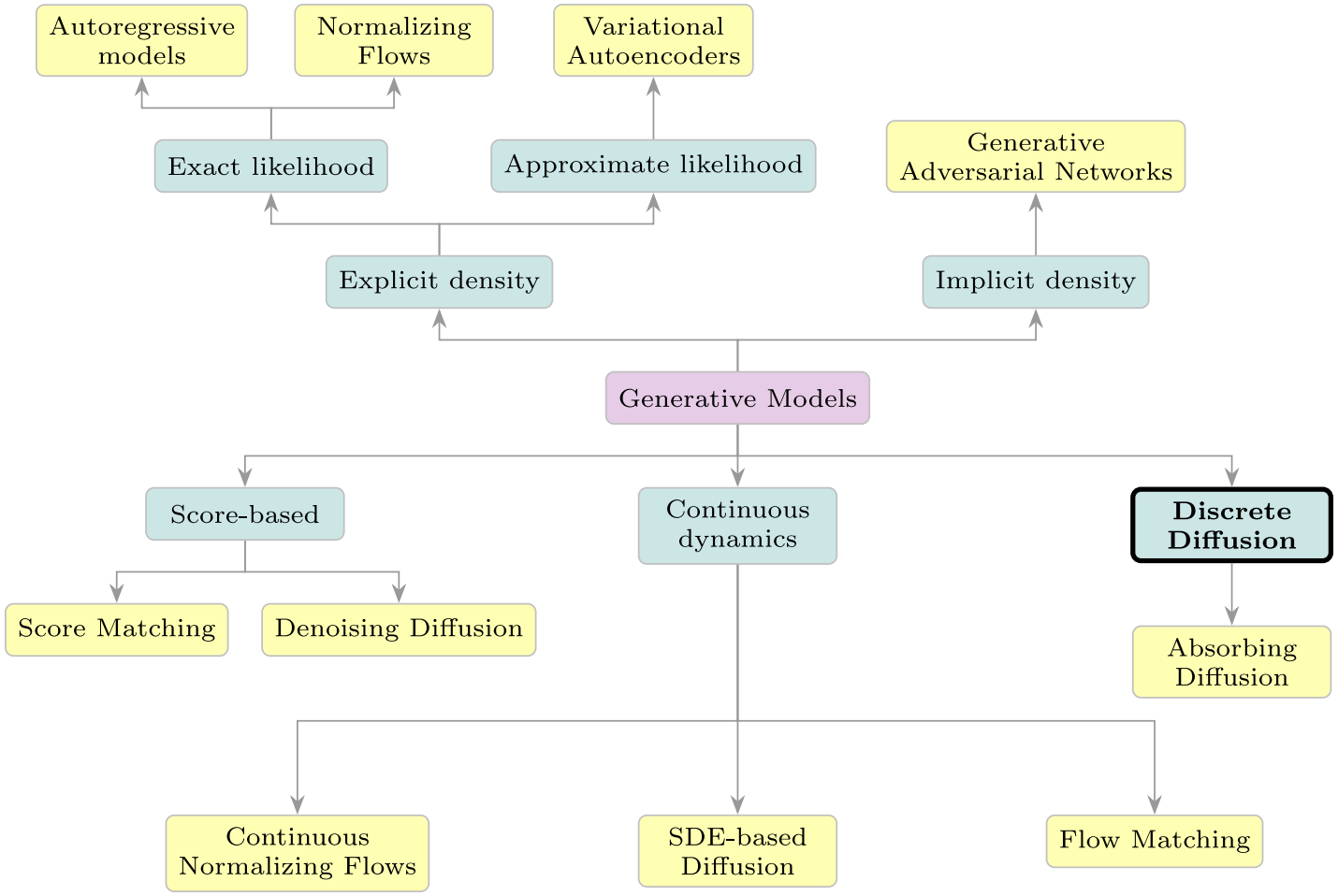
03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Generative Models Taxonomy



Discrete or Continuous Diffusion Models?

Reminder: Diffusion models define a forward corruption process and a reverse denoising process. Previously, we studied diffusion models with continuous states $\mathbf{x}(t) \in \mathbb{R}^m$.

Continuous state space

- **Discrete time** $t \in \{0, 1, \ldots, T\}$ $\Rightarrow$ **DDPM / NCSN**.
- **Continuous time** $t \in [0, 1]$ $\Rightarrow$ **Score-based SDE models**.

Discrete or Continuous Diffusion Models?

Reminder: Diffusion models define a forward corruption process and a reverse denoising process. Previously, we studied diffusion models with continuous states $\mathbf{x}(t) \in \mathbb{R}^m$.

Continuous state space

- **Discrete time** $t \in \{0, 1, \dots, T\}$ $\Rightarrow$ **DDPM / NCSN**.
- **Continuous time** $t \in [0, 1]$ $\Rightarrow$ **Score-based SDE models**.

Now we turn to diffusion over discrete-value states $\mathbf{x}(t) \in \{1, \dots, K\}^m$.

Discrete state space

- **Discrete time** $t \in \{0, 1, \dots, T\}$.
- **Continuous time** $t \in [0, 1]$.

Let's discuss why we need discrete diffusion models.

Why Discrete Diffusion Models?

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.
- **Bidirectional context:** use available context on both sides when predicting a token.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.
- **Bidirectional context:** use available context on both sides when predicting a token.
- **Flexible generation order:** choose which positions to generate next, including confidence-based ordering.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.
- **Bidirectional context:** use available context on both sides when predicting a token.
- **Flexible generation order:** choose which positions to generate next, including confidence-based ordering.
- **Infilling and controllability:** fill arbitrary gaps, preserve specified tokens, and steer sampling with guidance.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

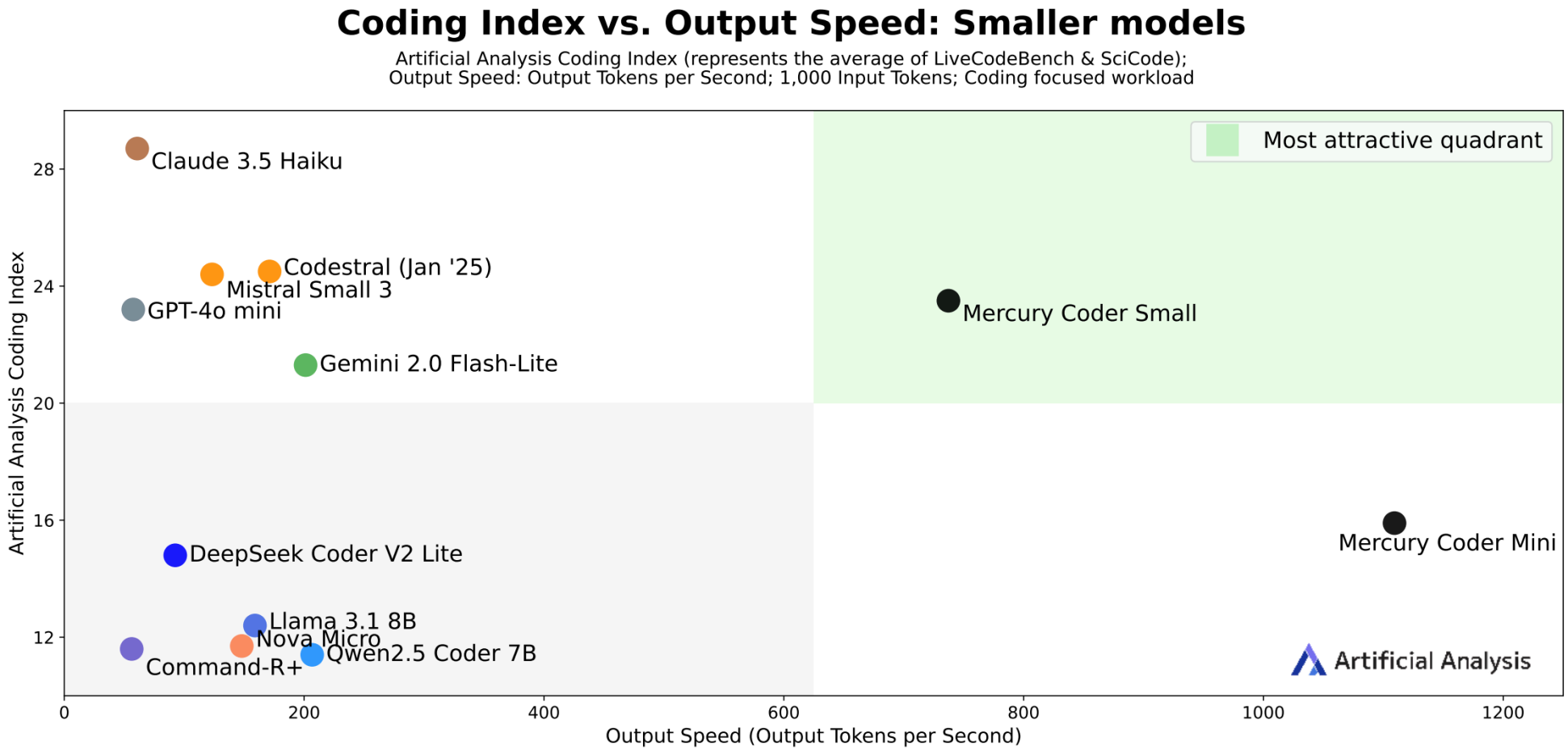
Why Discrete Diffusion Models?

Potential advantages over left-to-right autoregressive generation

- **Parallel generation:** generate multiple tokens per step, potentially reducing sequential decoding steps.
- **Bidirectional context:** use available context on both sides when predicting a token.
- **Flexible generation order:** choose which positions to generate next, including confidence-based ordering.
- **Infilling and controllability:** fill arbitrary gaps, preserve specified tokens, and steer sampling with guidance.
- **Iterative refinement:** revisit generated tokens when the sampler supports remasking or corrective updates.

Nie S. et al. Large language diffusion models, 2025. Wu C. et al. Fast-dLLM, 2025.
Schiff Y. et al. Simple guidance mechanisms for discrete diffusion models, 2025. Wang G. et al. Remasking discrete diffusion models with inference-time scaling, 2025.

2025 – Big Bang of Discrete Diffusion Models



Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

Discrete Diffusion Markov Chain

For discrete data, we instead define a Markov chain over categorical states:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{x}_{t-1}),$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

Discrete Diffusion Markov Chain

For discrete data, we instead define a Markov chain over categorical states:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{x}_{t-1}),$$

- Each $\mathbf{x}_t \in \{0,1\}^K$ is a **one-hot vector** encoding the categorical state (it is just one token).
- What is the transition matrix $\mathbf{Q}_t$?

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0, 1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1.$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1.$$

- As $t \to T$, the process drives the data toward a stationary distribution.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1.$$

- As $t \to T$, the process drives the data toward a stationary distribution.
- We design the transition matrices $\mathbf{Q}_t$ to achieve this behavior.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

Common choices

- **Uniform diffusion**

$$\mathbf{Q}_t = (1 - \beta_t)\mathbf{I} + \beta_t \mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

  Each token is replaced by a uniformly random symbol with probability β_t. The stationary distribution is uniform noise.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

Common choices

- **Uniform diffusion**

  $$\mathbf{Q}_t = (1 - \beta_t)\mathbf{I} + \beta_t \mathbf{U}, \qquad \mathbf{U}_{ij} = \tfrac{1}{K}.$$

  Each token is replaced by a uniformly random symbol with probability β_t. The stationary distribution is uniform noise.

- **Absorbing diffusion**

  $$\mathbf{Q}_t = (1 - \beta_t)\mathbf{I} + \beta_t \, \mathbf{e}_m \mathbf{1}^\top.$$

  Tokens are gradually replaced by a special mask m; the stationary distribution is fully masked.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Uniform Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Uniform Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

- Each token retains its original value with prob. $\bar{\alpha}_t$.
- It becomes uniformly random with prob. $(1-\bar{\alpha}_t)$.
- As $t \to T$, the process converges to the stationary uniform distribution.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

Absorbing Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\,\mathbf{e}_m\mathbf{1}^\top,$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\,\mathbf{I} + (1-\bar{\alpha}_t)\,\mathbf{e}_m\mathbf{1}^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

Absorbing Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\,\mathbf{e}_m\mathbf{1}^\top,$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\,\mathbf{I} + (1-\bar{\alpha}_t)\,\mathbf{e}_m\mathbf{1}^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

- Each token retains its original value with prob. $\bar{\alpha}_t$.
- It becomes $\mathbf{e}_m$ with prob. $(1-\bar{\alpha}_t)$.
- As $t \to T$, all tokens converge to the mask state: $q(\mathbf{x}_T) \approx \mathrm{Cat}(\mathbf{e}_m)$.
- This makes the process analogous to **masked language modeling**.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Uniform vs. Absorbing Transition Matrix

Aspect	Uniform Diffusion	Absorbing Diffusion
$\mathbf{Q}_t$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:t}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:\infty}$	$\mathbf{U}$	$\mathrm{Cat}(\mathbf{e}_m)$
Interpretation	Random replacement	Gradual masking of tokens
Application	Image diffusion	Text diffusion $\approx$ Masked LM

Uniform vs. Absorbing Transition Matrix

Aspect	Uniform Diffusion	Absorbing Diffusion
$\mathbf{Q}_t$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:t}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:\infty}$	$\mathbf{U}$	$\mathrm{Cat}(\mathbf{e}_m)$
Interpretation	Random replacement	Gradual masking of tokens
Application	Image diffusion	Text diffusion $\approx$ Masked LM

Observation

Both schemes gradually destroy information, but differ in their stationary limit. Absorbing diffusion bridges diffusion and masked-language-model objectives.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

Posterior of the Forward Process

ELBO

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

Posterior of the Forward Process

ELBO

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

- Conditioned reverse distribution $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ played crucial role in the continuous-state diffusion model.
- It shows the probability of a previous state given the noisy state $\mathbf{x}_t$ and the original clean data $\mathbf{x}_0$.

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)\, q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$
$$= \frac{\mathrm{Cat}(\mathbf{Q}_t) \cdot \mathrm{Cat}(\mathbf{Q}_{1:t-1})}{\mathrm{Cat}(\mathbf{Q}_{1:t})}.$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

Recall the ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big),$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Posterior of the Forward Process

Discrete conditioned reverse distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \text{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

Recall the ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\text{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big),$$

- Both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $q(\mathbf{x}_t|\mathbf{x}_0)$ are known analytically from the forward process.
- The reverse process $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is a learned categorical distribution:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \text{Cat}\big(\boldsymbol{\pi}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\big),$$

where $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ is a neural network.

Discrete-time ELBO for Discrete Diffusion

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big).$$

Discrete-time ELBO for Discrete Diffusion

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big).$$

Categorical KL

$$D_{\mathrm{KL}}\big(\mathrm{Cat}(\mathbf{q}) \,\|\, \mathrm{Cat}(\mathbf{p})\big) = \sum_{k=1}^{K} q_k \log \frac{q_k}{p_k} = \mathrm{H}(\mathbf{q}, \mathbf{p}) - \mathrm{H}(\mathbf{q}),$$

Discrete-time ELBO for Discrete Diffusion

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big).$$

Categorical KL

$$D_{\mathrm{KL}}\big(\mathrm{Cat}(\mathbf{q}) \,\|\, \mathrm{Cat}(\mathbf{p})\big) = \sum_{k=1}^{K} q_k \log \frac{q_k}{p_k} = \mathrm{H}(\mathbf{q}, \mathbf{p}) - \mathrm{H}(\mathbf{q}),$$

- $\mathrm{H}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\big)$ is a constant w.r.t. $\boldsymbol{\theta}$.
- $\mathrm{H}(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \log p_k$ is a **cross-entropy loss**.

Discrete-time ELBO for Discrete Diffusion

ELBO term

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big).$$

Categorical KL

$$D_{\mathrm{KL}}\big(\mathrm{Cat}(\mathbf{q}) \,\|\, \mathrm{Cat}(\mathbf{p})\big) = \sum_{k=1}^{K} q_k \log \frac{q_k}{p_k} = \mathrm{H}(\mathbf{q}, \mathbf{p}) - \mathrm{H}(\mathbf{q}),$$

- $\mathrm{H}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\big)$ is a constant w.r.t. $\boldsymbol{\theta}$.
- $\mathrm{H}(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \log p_k$ is a **cross-entropy loss**.

Therefore, minimizing $\mathcal{L}_t$ w.r.t. $\boldsymbol{\theta}$ is equivalent to minimizing

$$\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \mathrm{H}\Big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0),\, p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\Big).$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Outline

01 Two-Sided Conditioning

02 Link between Flow Matching and Score-Based Models

03 Discrete Diffusion Models

04 Forward Discrete Process

05 Reverse Diffusion Process

06 From Token To Sequence

From Token to Sequence

One-hot sequence representation

$$\mathbf{x}_t \in \{0, 1\}^K \quad \Leftrightarrow \quad \mathbf{X}_t \in \{0, 1\}^{K \times m}$$

Here $\mathbf{X}_t$ is a one-hot representation of a sequence of tokens.

From Token to Sequence

One-hot sequence representation

$$\mathbf{x}_t \in \{0,1\}^K \quad \Leftrightarrow \quad \mathbf{X}_t \in \{0,1\}^{K \times m}$$

Here $\mathbf{X}_t$ is a one-hot representation of a sequence of tokens.

Independent Token-wise Forward Process

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

- Each position i evolves according to its own Markov chain.
- Often the same transition matrix $\mathbf{Q}_t$ is shared across i.

From Token to Sequence

One-hot sequence representation

$$\mathbf{x}_t \in \{0,1\}^K \quad \Leftrightarrow \quad \mathbf{X}_t \in \{0,1\}^{K \times m}$$

Here $\mathbf{X}_t$ is a one-hot representation of a sequence of tokens.

Independent Token-wise Forward Process

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

- Each position i evolves according to its own Markov chain.
- Often the same transition matrix $\mathbf{Q}_t$ is shared across i.

Continuous Diffusion Analogy

- In Gaussian DDPMs with diagonal covariance, noise is independent per pixel.
- Structure is not in the noise; it is learned by the reverse model.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

From Token to Sequence

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

From Token to Sequence

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

$$q(\mathbf{X}_t|\mathbf{X}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{X}_0)$$

From Token to Sequence

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

$$q(\mathbf{X}_t|\mathbf{X}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{X}_0)$$

Conditioned Reverse Distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathrm{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

$$q(\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0) = \prod_{i=1}^{m} q(\mathbf{x}_{t-1}^i|\mathbf{x}_t^i, \mathbf{x}_0^i).$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

From Token to Sequence

$$q(\mathbf{X}_t|\mathbf{X}_{t-1}) = \prod_{i=1}^{m} q(\mathbf{x}_t^i|\mathbf{x}_{t-1}^i) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{X}_{t-1})$$

$$q(\mathbf{X}_t|\mathbf{X}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{X}_0)$$

Conditioned Reverse Distribution

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathrm{Cat}\left(\frac{\mathbf{Q}_t^\top \mathbf{x}_t \odot \mathbf{Q}_{1:t-1}\mathbf{x}_0}{\mathbf{x}_t^\top \mathbf{Q}_{1:t}\mathbf{x}_0}\right).$$

$$q(\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0) = \prod_{i=1}^{m} q(\mathbf{x}_{t-1}^i|\mathbf{x}_t^i, \mathbf{x}_0^i).$$

- All distributions defined by the forward process are factorized.
- Dependence appears in the learned reverse model.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Reverse Model for Sequence

$$p_{\boldsymbol{\theta}}(\mathbf{X}_{t-1}|\mathbf{X}_t) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}^i|\mathbf{X}_t).$$

- The output factorizes (parallel prediction across positions).
- Each factor conditions on the entire noisy sequence $\mathbf{X}_t$.
- This is exactly the **masked language modeling** pattern.

Reverse Model for Sequence

$$p_{\boldsymbol{\theta}}(\mathbf{X}_{t-1}|\mathbf{X}_t) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}^i|\mathbf{X}_t).$$

- The output factorizes (parallel prediction across positions).
- Each factor conditions on the entire noisy sequence $\mathbf{X}_t$.
- This is exactly the **masked language modeling** pattern.

Objective: $\mathcal{L}_t$ term

$$D_{\mathrm{KL}}\left(q(\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{X}_{t-1}|\mathbf{X}_t)\right)$$
$$= \sum_{i=1}^{m} D_{\mathrm{KL}}\left(q(\mathbf{x}_{t-1}^i|\mathbf{x}_t^i, \mathbf{x}_0^i) \| p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}^i|\mathbf{X}_t)\right).$$

Reverse Model for Sequence

Final objective: masked LM

$$\mathcal{L} = \sum_{t=1}^{T} \sum_{i=1}^{m} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \Big[-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0^i|\mathbf{X}_t) \Big].$$

Summary

- Two-sided conditioning uses pair conditioning $\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$ and yields the same procedure, but is more general (suitable for paired tasks).
- Diffusion and score-based models are special cases of the flow matching approach, but use curved trajectories.
- Discrete diffusion offers parallel generation and flexible conditioning; token revision requires a suitable sampler.
- Forward discrete diffusion process defines a Markov chain with discrete states; uniform and absorbing transitions make it tractable.
- Reverse discrete diffusion process uses the variational approach to invert forward process; the discrete-state ELBO is a cross-entropy loss.
- For sequences, the forward process factorizes over tokens, while the learned reverse model predicts all positions in parallel conditioning on the whole noisy sequence (masked LM pattern).